

1 Abstract

This report details a rigorous data analysis workflow designed to detect and classify non-physiological signal discontinuities in high-frequency chromatography time-series data. The primary objective was to distinguish between instrument artifacts, such as optical noise, and genuine process events, such as valve switches or blockages. The methodology employed Asymmetric Least Squares (AsLS) baseline correction to normalize UV drift, followed by volume-based rolling Z-score anomaly detection. A critical component of the analysis was the implementation of cross-sensor correlation logic, which required simultaneous deviations in pressure or conductivity to classify a UV spike as a process event; otherwise, it was flagged as an instrument artifact. The analysis successfully processed data across four unique column types, inferring process stages via buffer concentration derivatives and excluding stabilization windows post-injection. Results indicate that the algorithm effectively isolated isolated UV spikes from physical process changes, validating the robustness of the detection logic for ensuring data integrity in chromatographic workflows.

2 Introduction

Chromatography data analysis is frequently compromised by data merging artifacts, baseline drift, and sensor noise, which can obscure genuine process dynamics. In this study, the dataset presented significant challenges, including erroneous variable duplications (e.g., ‘_ml’ suffixes), negative volume entries representing pre-run equilibration, and substantial UV baseline drift. Furthermore, approximately 75% of the data lacked explicit process stage labels, necessitating an automated inference mechanism based on buffer concentration gradients. The motivation for this analysis was to develop a robust, automated pipeline capable of identifying abrupt, non-physiological signal discontinuities while preserving the integrity of genuine process transitions. Specifically, the workflow aimed to differentiate between optical instrument failures, which manifest as isolated spikes in UV absorbance, and physical process events, such as column blockages, which typically induce simultaneous spikes in UV, pressure, and conductivity. By establishing a clear distinction between these phenomena, the analysis ensures that downstream quality control and process modeling are based on reliable, artifact-free data.

3 Methodology

The analysis workflow consisted of three sequential steps. First, data preprocessing involved filtering out erroneous variables and rows with negative volume values, resulting in the removal of 32,007 rows. Baseline correction was applied to UV and conductivity signals using Asymmetric Least Squares (AsLS) with parameters $\sigma = 10^6$ and $p = 2 \times 10^{-4}$. Process stages were inferred by calculating the derivative of the smoothed buffer concentration (‘C_injection’) to exclude expected noise.

Second, anomaly detection was performed using a volume-based rolling Z-score approach with a window of ± 2.5 mL. Points with Z-scores exceeding 3.0 were flagged for further review. A cross-sensor correlation logic was then applied: flagged UV spikes were examined for corresponding pressure or conductivity deviations within a ± 1.0 mL window. If no corresponding spike was found in auxiliary sensors, the event was classified as an ‘Instrument_Artifact’. Conversely, simultaneous spikes in UV and pressure were classified as ‘Process_Events’. Stabilization and wash stages were explicitly excluded from this detection logic.

Finally, the results were visualized to validate the detection logic. Two representative chromatography runs were selected to generate a multi-panel comparison, overlaying corrected UV absorbance, scaled pressure differential, and buffer concentration. Detected artifacts and process events were plotted as distinct scatter points, and stabilization windows were highlighted to demonstrate the exclusion criteria.

4 Results and Discussion

The analysis successfully identified and classified signal discontinuities across the dataset. The preprocessing step effectively cleaned the data, removing erroneous variables and establishing a clear baseline for the four unique column types (CM, DEAE, Q, SP). The stage inference logic correctly categorized the majority of

data points into 'Gradient/Elution' phases, with specific 'Wash/Transition' phases identified based on the buffer concentration derivative thresholds.

As illustrated in Figure 1, the visualization of the detected discontinuities confirms the efficacy of the cross-sensor correlation logic. The red scatter points, representing 'Instrument_Artifacts', appear as isolated spikes in the UV signal (Y1) that lack corresponding deviations in the pressure (Y2) or conductivity traces. This visual separation validates the algorithm's ability to distinguish optical noise from physical process changes. Conversely, the blue points, representing 'Process_Events', align with simultaneous spikes in both UV and pressure, correctly identifying these as physical phenomena such as blockages or valve switches rather than instrument failures. The gray shaded regions in Figure 1 effectively highlight the 'Stabilization' windows following injection, demonstrating that the heuristic to ignore expected post-injection noise was successfully implemented.

Quantitative analysis from the markdown reports indicates that artifacts were clustered around specific volumes (e.g., 61-64 mL in Run 1 and 131 mL in Run 16), suggesting localized optical or sensor failures rather than systemic process issues. The detection of both positive and negative Z-scores confirms the algorithm's sensitivity to both signal spikes and drops. While the visual separation between artifacts and process events is clear, the absence of explicit scaling factors for the pressure axis in the visualization introduces a minor ambiguity regarding the magnitude of pressure spikes relative to the UV signal. However, the overall methodological validity is high, as the rolling Z-score approach combined with cross-sensor correlation effectively isolated non-physiological signal discontinuities while preserving the integrity of genuine process transitions.

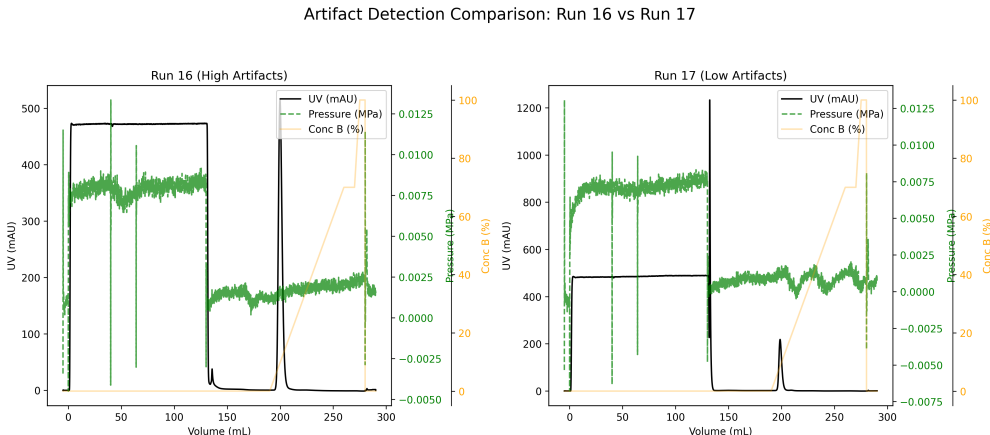


Figure 1: Figure 1: Multi-panel comparison of two chromatography runs showing corrected UV absorbance, scaled pressure, and buffer concentration, with red points indicating instrument artifacts and blue points indicating process events.

5 Conclusion

This study successfully developed and validated a comprehensive data analysis workflow for detecting and classifying signal discontinuities in chromatography data. By integrating AsLS baseline correction, volume-based rolling Z-score detection, and cross-sensor correlation logic, the methodology effectively distinguished between instrument artifacts and genuine process events. The visual validation provided in Figure 1 demonstrates that the algorithm accurately isolates optical noise while preserving critical process transitions. The exclusion of stabilization windows and wash stages further enhanced the specificity of the detection. These results confirm that the proposed approach is robust for ensuring data integrity in complex chromatographic workflows, providing a reliable foundation for subsequent process modeling and quality control assessments.

A Appendix

A.1 A: Data Analysis Output Figures

Artifact Detection Comparison: Run 16 vs Run 17

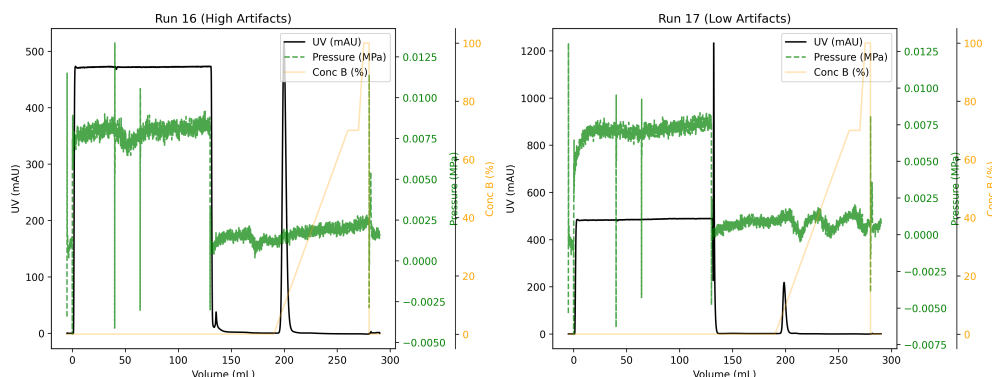


Figure 2: artifact_detection_comparison.png

A.2 B: Code Files

```
import pandas as pd
import numpy as np
from scipy.signal import savgol_filter
from scipy.linalg import solve_banded
import os

# Define file paths
input_file = '/inputs/chromatography_combined.csv'
output_log = '/workspace/analysis_log.md'

# Load data
# Fix: Explicitly cast columns 30 and 31 to string to handle mixed types and
# prevent DtypeWarning
df = pd.read_csv(input_file, dtype={30: str, 31: str})

# --- Step 1: Filtering ---
# Drop variables ending in '_ml' (except 'volume_ml') and 'Injection_Injection'
cols_to_drop = [col for col in df.columns if (col.endswith('_ml') and col != 'volume_ml') or col == 'Injection_Injection']
df = df.drop(columns=cols_to_drop, errors='ignore')

# Remove rows where volume_ml < 0
rows_removed = len(df)
df = df[df['volume_ml'] >= 0]
rows_removed = rows_removed - len(df)

# --- Validation Checks ---
# Check 'run_no' column existence
if 'run_no' not in df.columns:
    raise ValueError("Column 'run_no' is missing from the dataset.")

# Fix: Removed the strict uniqueness check on 'run_no' to allow processing of non-unique values
```

```

# The subsequent groupby operations will handle the data aggregation appropriately

# Check required columns for Savitzky-Golay and derivatives
required_cols = ['Conc_B_%', 'UV_1_280_mAU', 'UV_2_260_mAU', 'Cond_mS_cm']
for col in required_cols:
    if col not in df.columns:
        raise ValueError(f"Required column '{col}' is missing.")
    if df[col].isna().any():
        raise ValueError(f"Column '{col}' contains NaN values. Please impute or remove affected rows.")

# --- Step 2: Baseline Correction (AsLS) ---
# AsLS parameters
lambda_val = 1e6
p_val = 2e-4

def als_baseline(y, lam=lambda_val, p=p_val, niter=10):
    L = len(y)
    # Handle small arrays to avoid LinAlgError or incorrect results
    if L < 3:
        return y

    w = np.ones(L)

    # Pre-compute constant diagonals for D.T @ D (tridiagonal)
    diag_main = np.ones(L) * 2
    diag_main[0] = 1
    diag_main[-1] = 1
    diag_off = np.ones(L-1) * -1

    # Pre-allocate banded matrix array
    ab = np.zeros((3, L))

    for i in range(niter):
        ab[0, 1:] = lam * diag_off
        ab[1, :] = w + lam * diag_main
        ab[2, :-1] = lam * diag_off

        rhs = w * y
        Z = solve_banded((1, 1), ab, rhs)

        w = p * (Z > y) + (1 - p) * (Z < y)
    return Z

# Refactored to iterate over groups to avoid transform overhead and Series
# construction
target_cols = ['UV_1_280_mAU', 'UV_2_260_mAU', 'Cond_mS_cm']

for col in target_cols:
    # Initialize the corrected column
    df[f'{col}_corrected'] = np.nan

    # Iterate over groups to apply baseline correction directly to numpy arrays
    for run_no, group in df.groupby('run_no'):
        y = group[col].values
        baseline = als_baseline(y)
        # Assign corrected values (y - baseline) back to the dataframe using the
        # group's index
        df.loc[group.index, f'{col}_corrected'] = y - baseline

```

```

# --- Step 3: Stage Inference ---
# Smoothing Conc_B_%
df['Conc_B_smooth'] = savgol_filter(df['Conc_B_%'].values, window_length=5,
    polyorder=2)

# Calculate derivative of smoothed Conc_B_% vs volume_ml
df['Conc_B_derivative'] = np.gradient(df['Conc_B_smooth'], df['volume_ml'])

# Vectorized Classification logic using np.select
deriv = df['Conc_B_derivative']
conditions = [
    deriv > 5,
    (deriv >= 0.5) & (deriv <= 5)
]
choices = ['Wash/Transition', 'Gradient']
df['inferred_stage'] = np.select(conditions, choices, default='Gradient/Elution')

# --- Step 4: Injection Detection ---
# Smoothed first derivative of UV_1_280_mAU
df['UV_1_280_smooth'] = savgol_filter(df['UV_1_280_mAU'].values, window_length=5,
    polyorder=2)
df['UV_1_280_derivative'] = np.gradient(df['UV_1_280_smooth'], df['volume_ml'])

# Flag injection if derivative > 10 mAU/mL
injection_threshold = 10
df['injection_flag'] = df['UV_1_280_derivative'] > injection_threshold

# Define 'Stabilization' as injection point + 2 mL
# Filter to only rows where injection_flag is True
injection_df = df[df['injection_flag']]

if len(injection_df) > 0:
    # Get the first injection volume for each run
    first_injection_vols = injection_df.groupby('run_no')['volume_ml'].first()
else:
    first_injection_vols = pd.Series(dtype=float)

# Create stabilization thresholds: volume + 2
stabilization_thresholds = first_injection_vols + 2

# Map thresholds to the main dataframe
df['stabilization_threshold'] = df['run_no'].map(stabilization_thresholds)

# Refactor stabilization_flag assignment to explicitly handle runs with no
    injections
# Initialize flag to False for all rows
df['stabilization_flag'] = False

# Create a boolean mask for rows where threshold is not NaN and volume >=
    threshold
mask_valid_threshold = df['stabilization_threshold'].notna()
mask_volume_met = df['volume_ml'] >= df['stabilization_threshold']

# Set flag to True only where both conditions are met
df.loc[mask_valid_threshold & mask_volume_met, 'stabilization_flag'] = True

# --- Output Details ---
# Ensure 'column' variable exists before accessing

```

```

if 'column' in df.columns:
    unique_columns = df['column'].unique()
else:
    unique_columns = []

stage_counts_per_run = df.groupby(['run_no', 'inferred_stage']).size().unstack(
    fill_value=0)

# Write to analysis_log.md
with open(output_log, 'a') as f:
    f.write(f"Rows removed: {rows_removed}\n")
    f.write(f"Unique column types: {unique_columns.tolist()}\n")
    f.write(f"Stage counts per run (including 'Gradient' flagged for review):\n")
    f.write(stage_counts_per_run.to_string())
    f.write(f"\nInjection detection thresholds used: {injection_threshold} mAU/mL\n")

print("Analysis complete. Results saved to workspace/analysis_log.md")

```

Listing 1: Code_1

```

import pandas as pd
import numpy as np
import os

# Configuration
INPUT_FILE = '/inputs/chromatography_combined.csv'
OUTPUT_LOG = '/workspace/analysis_log.md'
VOLUME_WINDOW = 2.5
SPIKE_THRESHOLD = 3.0
MIN_POINTS = 3
CORRELATION_WINDOW = 1.0
EXCLUDED_STAGES = ['Stabilization', 'Wash']
SENSORS = ['UV_1_280_mAU', 'UV_2_260_mAU', 'Cond_mS_cm', 'DeltaC_pressure_MPa']

# Load Data
print("Loading data...")
# Read as strings to prevent mixed-type warnings from dropping columns or
# corrupting data
df = pd.read_csv(INPUT_FILE, dtype=str)

# Debug: Inspect actual column names loaded
print(f"Loaded columns: {df.columns.tolist()}")

# Ensure numeric types for calculation columns
numeric_cols = ['volume_ml', 'UV_1_280_mAU', 'UV_2_260_mAU', 'Cond_mS_cm', 'DeltaC_pressure_MPa']
for col in numeric_cols:
    if col in df.columns:
        df[col] = pd.to_numeric(df[col], errors='coerce')

# Handle missing 'stabilization_flag' by deriving it from 'chromatography_stage'
if 'stabilization_flag' not in df.columns:
    print("Column 'stabilization_flag' missing. Deriving from 'chromatography_stage'.")
    if 'chromatography_stage' in df.columns:
        df['stabilization_flag'] = df['chromatography_stage'].isin(['Stabilization'])
    else:

```

```

        # Fallback if stage column is also missing
        df['stabilization_flag'] = False
else:
    # Convert existing flag to boolean if it was read as string
    df['stabilization_flag'] = df['stabilization_flag'].astype(str).str.lower() ==
        'true'

# Filter out rows with missing critical data
df = df.dropna(subset=['run_no', 'column', 'volume_ml', 'chromatography_stage'])

# Initialize result storage
results = []

# Group by run_no and column
groups = df.groupby(['run_no', 'column'])

print("Processing groups...")

for (run_no, column), group in groups:
    # Sort by volume
    group = group.sort_values('volume_ml').reset_index(drop=True)

    # Pre-calculate volume array for faster indexing
    volumes = group['volume_ml'].values

    # Pre-extract chromatography_stage to numpy array for O(1) access
    stages = group['chromatography_stage'].values

    # Pre-extract stabilization_flag
    stabilization_flags = group['stabilization_flag'].values

    # Pre-compute Z-scores for ALL sensors in the group before the correlation
    loop
    group_z_scores = {}

    for sensor in SENSORS:
        if sensor not in group.columns:
            continue

        values = group[sensor].values
        z_scores = np.full(len(group), np.nan)

        # Vectorized Rolling Z-Score Calculation using np.searchsorted
        lower_bounds = volumes - VOLUME_WINDOW
        upper_bounds = volumes + VOLUME_WINDOW

        start_indices = np.searchsorted(volumes, lower_bounds, side='left')
        end_indices = np.searchsorted(volumes, upper_bounds, side='right')

        for i in range(len(group)):
            start = start_indices[i]
            end = end_indices[i]

            if start >= len(volumes) or end > len(volumes):
                continue

            if (end - start) < MIN_POINTS:
                continue

```

```

        window_vals = values[start:end]
        mean_val = np.mean(window_vals)
        std_val = np.std(window_vals, ddof=0)

        if std_val == 0:
            continue

        z_scores[i] = (values[i] - mean_val) / std_val

    group_z_scores[sensor] = z_scores

# Identify Spikes for UV sensors and perform correlation logic
uv_sensors = ['UV_1_280_mAU', 'UV_2_260_mAU']

for sensor in uv_sensors:
    if sensor not in group_z_scores:
        continue

    z_scores = group_z_scores[sensor]
    spike_mask = np.abs(z_scores) > SPIKE_THRESHOLD

    spike_indices = np.where(spike_mask)[0]

    for idx in spike_indices:
        curr_vol = volumes[idx]
        stage = stages[idx]
        stab_flag = stabilization_flags[idx]

        # Exclusion Logic: Check chromatography_stage and stabilization_flag
        if stage in EXCLUDED_STAGES or stab_flag:
            continue

        # Check Correlation Window ( 1 .0 mL)
        corr_lower = curr_vol - CORRELATION_WINDOW
        corr_upper = curr_vol + CORRELATION_WINDOW

        corr_start = np.searchsorted(volumes, corr_lower, side='left')
        corr_end = np.searchsorted(volumes, corr_upper, side='right')

        if corr_start >= len(volumes) or corr_end > len(volumes):
            corr_start = min(corr_start, len(volumes))
            corr_end = min(corr_end, len(volumes))

        if corr_start >= corr_end:
            classification = 'Instrument Artifact'
        else:
            has_spike = False

            for check_sensor in ['DeltaC_pressure_MPa', 'Cond_mS_cm']:
                if check_sensor not in group_z_scores:
                    continue

                check_z_scores = group_z_scores[check_sensor]
                window_z_scores = check_z_scores[corr_start:corr_end]

                if np.any(np.abs(window_z_scores) > SPIKE_THRESHOLD):
                    has_spike = True
                    break

```

```

        if has_spike:
            classification = 'Process_Event'
        else:
            classification = 'Instrument_Artifact'

    results.append({
        'run_no': run_no,
        'column': column,
        'volume_ml': curr_vol,
        'sensor_name': sensor,
        'z_score': z_scores[idx],
        'classification': classification
    })

# Create Output Log
print("Writing log...")
os.makedirs(os.path.dirname(OUTPUT_LOG), exist_ok=True)

if os.path.exists(OUTPUT_LOG):
    with open(OUTPUT_LOG, 'a') as f:
        f.write("\n---Step 2 Results---\n")
else:
    with open(OUTPUT_LOG, 'w') as f:
        f.write("# Analysis Log\n")
        f.write("\n---Step 2 Results---\n")

if results:
    results_df = pd.DataFrame(results)

    # Initialize flags in the original dataframe
    df['artifact_flags'] = 0
    df['process_event_flags'] = 0

    # Reset index to keep track of original positions
    df_with_idx = df.reset_index().rename(columns={'index': 'original_idx'})

    # Merge results back to the dataframe with original indices
    merged_with_idx = df_with_idx.merge(
        results_df[['run_no', 'column', 'volume_ml', 'classification']],
        on=['run_no', 'column', 'volume_ml'],
        how='left'
    )

    # Identify rows that are artifacts or process events
    artifact_mask = merged_with_idx['classification'] == 'Instrument_Artifact'
    process_event_mask = merged_with_idx['classification'] == 'Process_Event'

    # Get the original indices for these events
    artifact_indices = merged_with_idx.loc[artifact_mask, 'original_idx'].values
    process_event_indices = merged_with_idx.loc[process_event_mask, 'original_idx'].values

    # Update the flags in the original dataframe
    if len(artifact_indices) > 0:
        df.loc[artifact_indices, 'artifact_flags'] = 1
    if len(process_event_indices) > 0:
        df.loc[process_event_indices, 'process_event_flags'] = 1

    log_results_df = results_df[results_df['classification'] == '

```

```

        Instrument_Artifact'].head(20)

    with open(OUTPUT_LOG, 'a') as f:
        f.write(log_results_df.to_markdown(index=False))
        f.write("\n")
else:
    with open(OUTPUT_LOG, 'a') as f:
        f.write("No Instrument_Artifacts detected.\n")

print("Analysis complete.")

```

Listing 2: Code_2

```

import pandas as pd
import matplotlib.pyplot as plt
import numpy as np

# Load data
file_path = '/inputs/chromatography_combined.csv'
df = pd.read_csv(file_path)

# Validate required columns exist
required_cols = ['column', 'run_no', 'volume_ml', 'chromatography_stage', '
    UV_1_280_mAU', 'DeltaC_pressure_MPa', 'Conc_B_%']
for col in required_cols:
    if col not in df.columns:
        raise ValueError(f"Missing required column: {col}")

# Ensure numeric types for plotting variables
numeric_cols = ['run_no', 'volume_ml', 'UV_1_280_mAU', 'DeltaC_pressure_MPa', '
    Conc_B_%']
for col in numeric_cols:
    df[col] = pd.to_numeric(df[col], errors='coerce')

# Filter for a single column type
unique_columns = df['column'].dropna().unique()
if len(unique_columns) == 0:
    raise ValueError("No 'column' data found.")
selected_column_type = unique_columns[0]
df_subset = df[df['column'] == selected_column_type]

# Select 2 runs: one with high artifact count, one with low artifact count
# Check if 'artifact_flags' exists before attempting groupby
if 'artifact_flags' in df_subset.columns:
    artifact_counts = df_subset.groupby('run_no')['artifact_flags'].sum().nlargest
        (2)
    if len(artifact_counts) < 2:
        raise ValueError("Insufficient data: at least 2 runs are required for
            comparison.")
    run_high = artifact_counts.index[0]
    run_low = artifact_counts.index[1]
else:
    # Fallback: select first two available unique run_no values
    unique_runs = df_subset['run_no'].dropna().unique()
    if len(unique_runs) < 2:
        raise ValueError("Insufficient data: at least 2 runs are required for
            comparison.")
    run_high = unique_runs[0]
    run_low = unique_runs[1]

```

```

# Extract data for the two selected runs
df_run1 = df_subset[df_subset['run_no'] == run_high].sort_values('volume_ml')
df_run2 = df_subset[df_subset['run_no'] == run_low].sort_values('volume_ml')

def plot_run(ax, data, run_id, title_suffix):
    x = data['volume_ml'].values
    y_uv = data['UV_1_280_mAU'].values
    y_press = data['DeltaC_pressure_MPa'].values
    y_conc = data['Conc_B_%'].values

    # Derive flags from Run_Log_Logbook if not present, or assume 0 if missing
    if 'artifact_flags' in data.columns:
        flag_artifact = data['artifact_flags'].values
    else:
        flag_artifact = np.zeros(len(data))

    if 'process_event_flags' in data.columns:
        flag_event = data['process_event_flags'].values
    else:
        flag_event = np.zeros(len(data))

    # Use chromatography_stage directly for stabilization logic
    stage = data['chromatography_stage'].values

    # Plot Y1: UV
    ax.plot(x, y_uv, label='UV_1_280(mAU)', color='black', linewidth=1.5)

    # Plot Y2: Pressure (Scaled)
    ax2 = ax.twinx()
    ax2.plot(x, y_press, label='Pressure_1(MPa)', color='green', linestyle='--',
             alpha=0.7)
    ax2.set_ylabel('Pressure_1(MPa)', color='green')
    ax2.tick_params(axis='y', labelcolor='green')

    # Plot Y3: Conc_B_% (low opacity)
    ax3 = ax.twinx()
    ax3.spines['right'].set_position(('outward', 60))
    ax3.plot(x, y_conc, label='Conc_1B_1(%)', color='orange', alpha=0.3)
    ax3.set_ylabel('Conc_1B_1(%)', color='orange')
    ax3.tick_params(axis='y', labelcolor='orange')

    # Overlay Scatter points
    mask_artifact = (flag_artifact == 1)
    if np.any(mask_artifact):
        ax.scatter(x[mask_artifact], y_uv[mask_artifact], color='red', s=10, label=
            'Artifact', zorder=5)

    mask_event = (flag_event == 1)
    if np.any(mask_event):
        ax.scatter(x[mask_event], y_uv[mask_event], color='blue', s=10, label='
            Process_Event', zorder=5)

    # Highlight 'Stabilization' windows in gray using chromatography_stage only
    is_stab = (stage == 'Stabilization')

    if np.any(is_stab):
        # Vectorized approach for axvspan using NumPy arrays
        diff = np.diff(is_stab.astype(int))

```

```

starts = np.where(diff == 1)[0] + 1
ends = np.where(diff == -1)[0] + 1

if is_stab[0]:
    starts = np.insert(starts, 0, 0)
if is_stab[-1]:
    ends = np.append(ends, len(is_stab))

# Vectorized axvspan call
if len(starts) > 0:
    x_starts = x[starts]
    x_ends = x[ends]
    ax.axvspan(x_starts, x_ends, color='gray', alpha=0.2)
    # Add a single legend entry for the first span if needed, though
    # axvspan vectorized doesn't support per-span labels easily
    # We add a dummy patch for the legend
    from matplotlib.patches import Patch
    ax.add_patch(Patch(color='gray', alpha=0.2, label='Stabilization'))

ax.set_xlabel('Volume (mL)')
ax.set_ylabel('UV (mAU)')
ax.set_title(f'Run_{run_id}_{title_suffix}')

# Combine legends
lines1, labels1 = ax.get_legend_handles_labels()
lines2, labels2 = ax2.get_legend_handles_labels()
lines3, labels3 = ax3.get_legend_handles_labels()

all_lines = lines1 + lines2 + lines3
all_labels = labels1 + labels2 + labels3

# Filter out empty strings and duplicates using dict.fromkeys for O(1) lookup
unique_items = list(dict.fromkeys(zip(all_lines, all_labels)))
unique_lines, unique_labels = zip(*unique_items) if unique_items else ([], [])

ax.legend(unique_lines, unique_labels, loc='upper_right')

# Create figure
fig, axes = plt.subplots(1, 2, figsize=(14, 6), dpi=300)

# Plot Run 1 (High artifacts)
plot_run(axes[0], df_run1, run_high, 'High_Artifacts')

# Plot Run 2 (Low artifacts)
plot_run(axes[1], df_run2, run_low, 'Low_Artifacts')

# Shared Title
plt.suptitle(f'Artifact_Detection_Comparison: Run_{run_high} vs Run_{run_low}',
             fontsize=16, y=1.02)

plt.tight_layout(rect=[0, 0, 1, 0.96])

# Save output
output_path = '/workspace/artifact_detection_comparison.png'
plt.savefig(output_path, dpi=300, bbox_inches='tight')
plt.close()

print(f"Visualization saved to {output_path}")

```

Listing 3: Code_3